Evidence: CI/CD pipeline, failure test and rollback test

Screenshots from my own runs of the Library API pipeline.

Figure 1. App running after the first release

localhost:8080 after the first deployment, logged in as the demo admin.

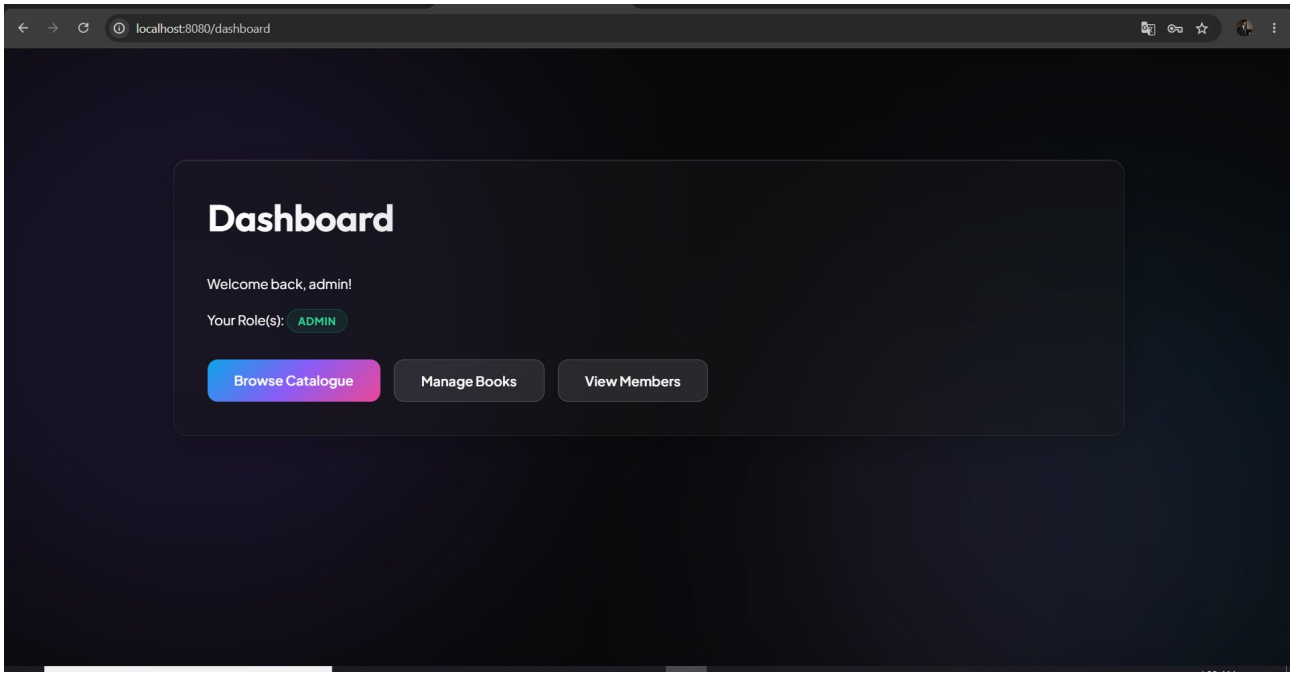


Figure 2. First release run

Release v1.2, run #9. All three jobs passed. Total time 3m 37s.

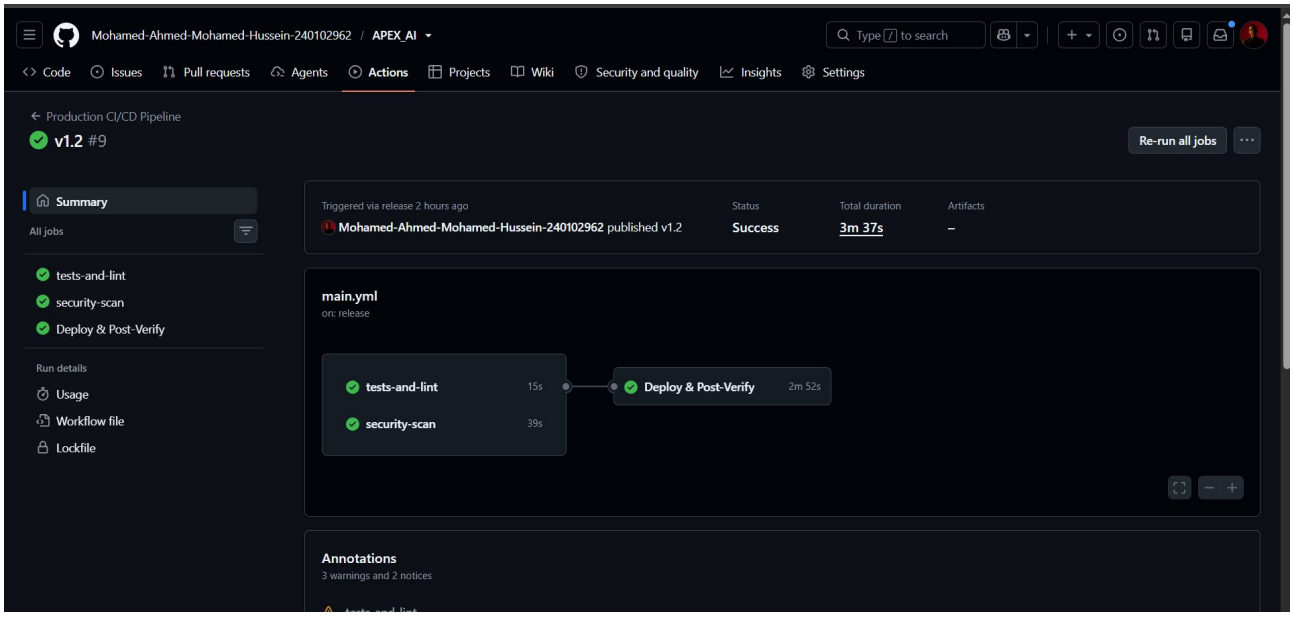
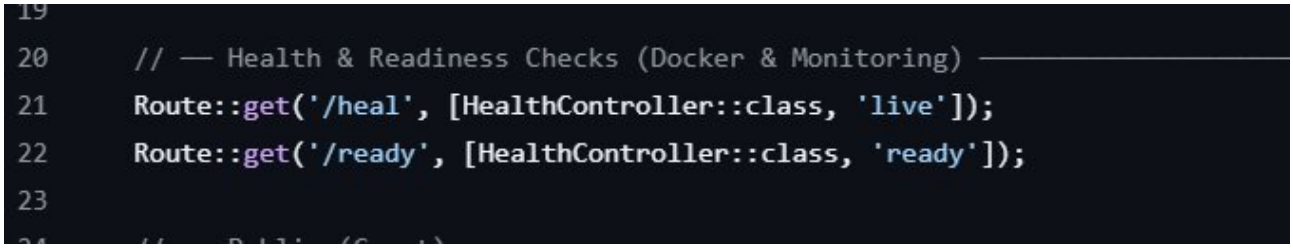


Figure 3. Intentional failure: broken route

I changed /health to /heal in routes/api.php on purpose.



```
19
20      // — Health & Readiness Checks (Docker & Monitoring) ——————————
21      Route::get('/heal', [HealthController::class, 'live']);
22      Route::get('/ready', [HealthController::class, 'ready']);
23
```

Figure 4. Intentional failure: result

Run #11. tests-and-lint failed and the deploy job did not run.

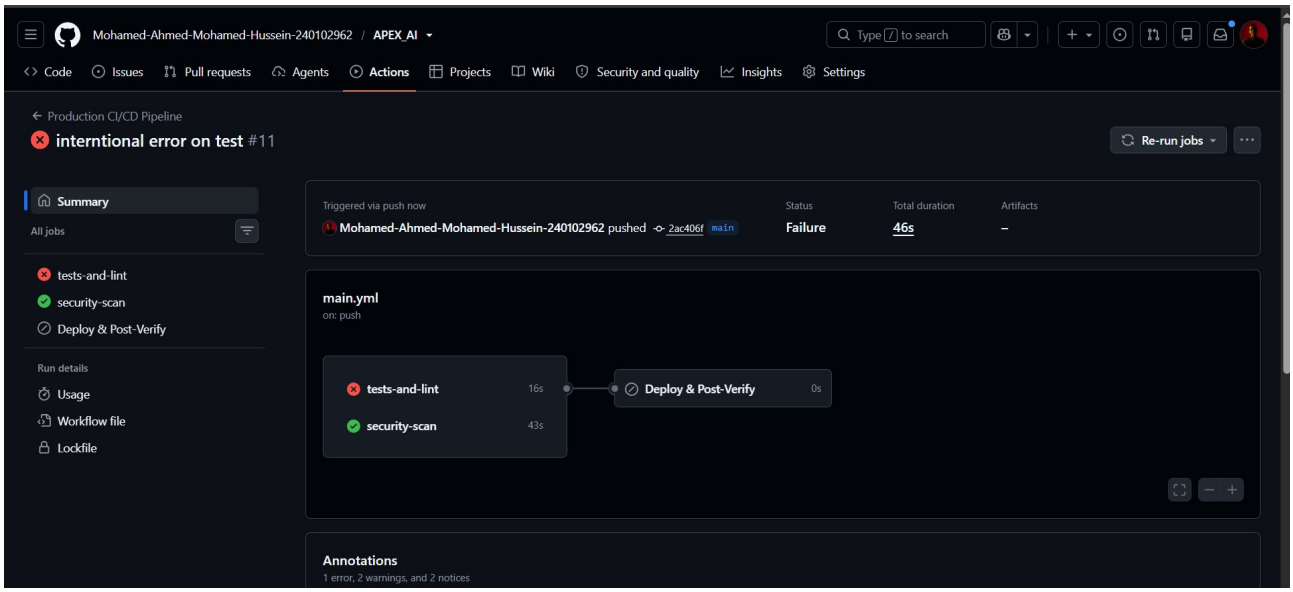


Figure 5. After fixing the route

Run #12. Tests and scan passed again. The deploy only runs on a release.

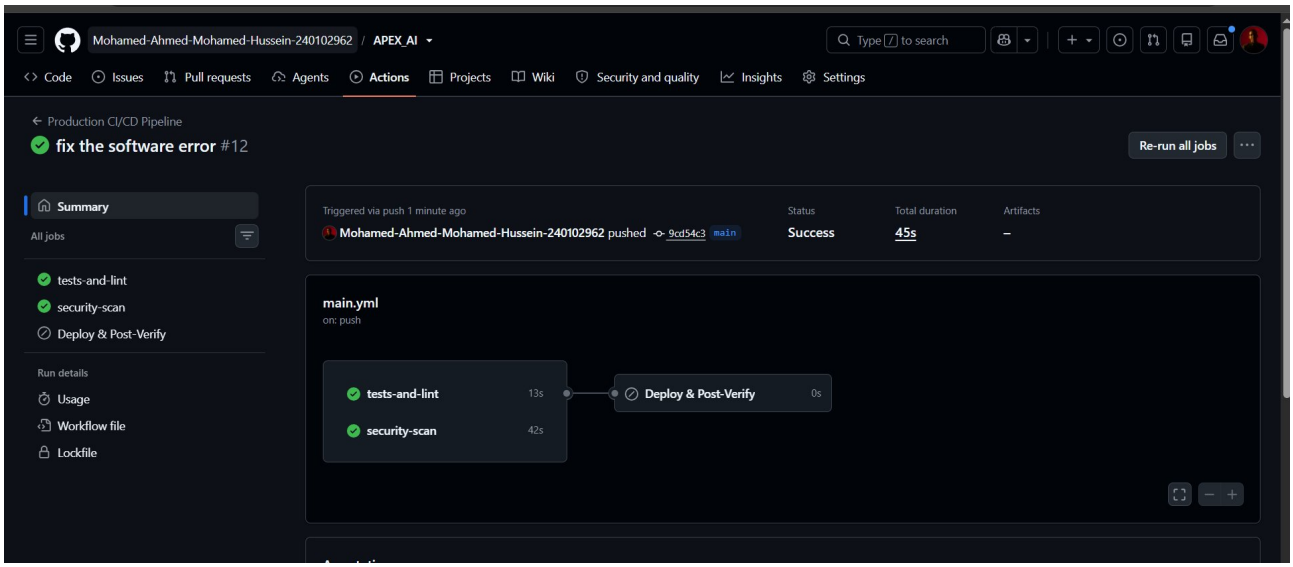


Figure 6. False release

New release v1.3 created to test the rollback.

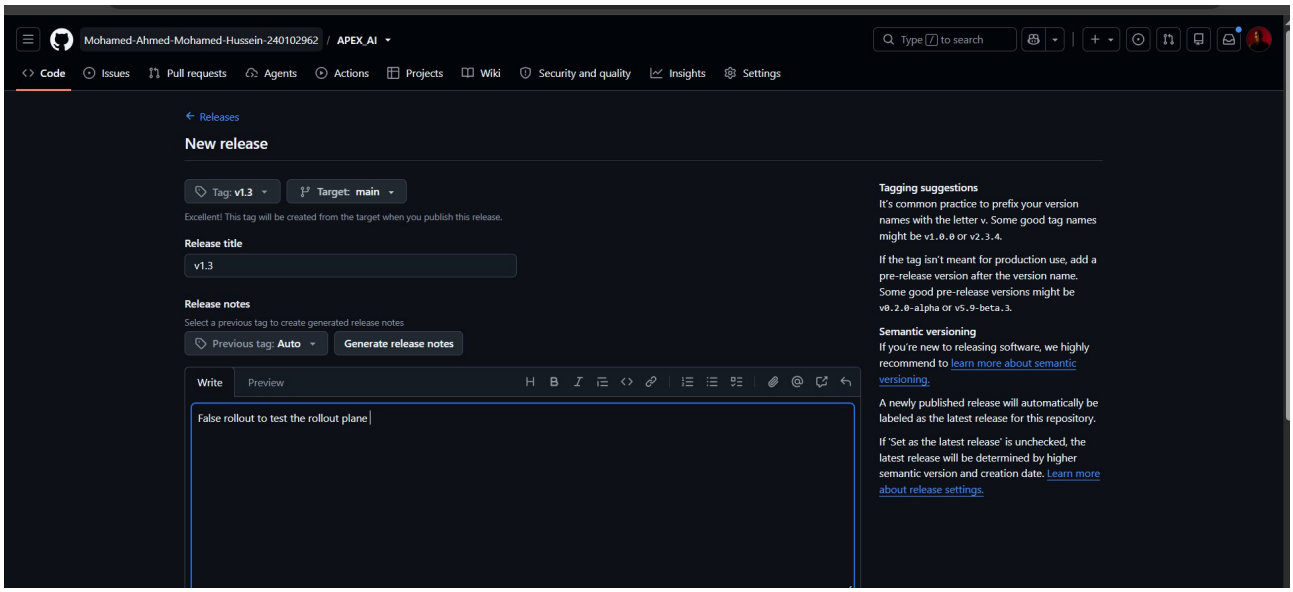


Figure 7. False release: deploy failed

Release v1.3, run #14. Tests and scan passed, Deploy & Post-Verify failed (curl exit code 22).

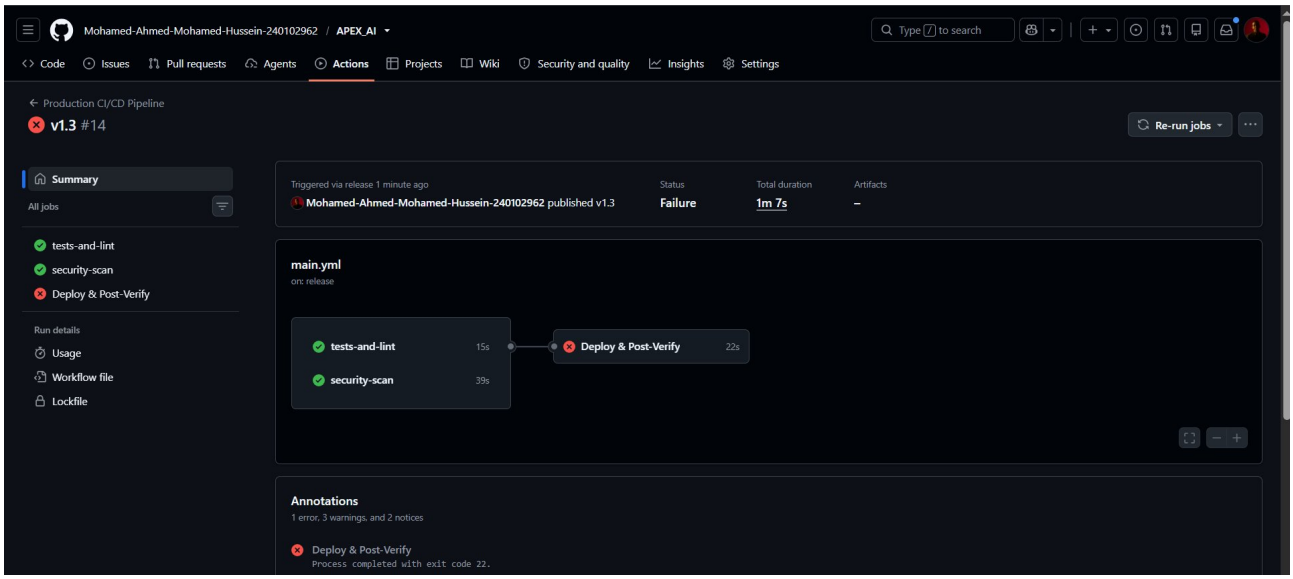


Figure 8. App still available
localhost:8080 still answers after the failed deployment.

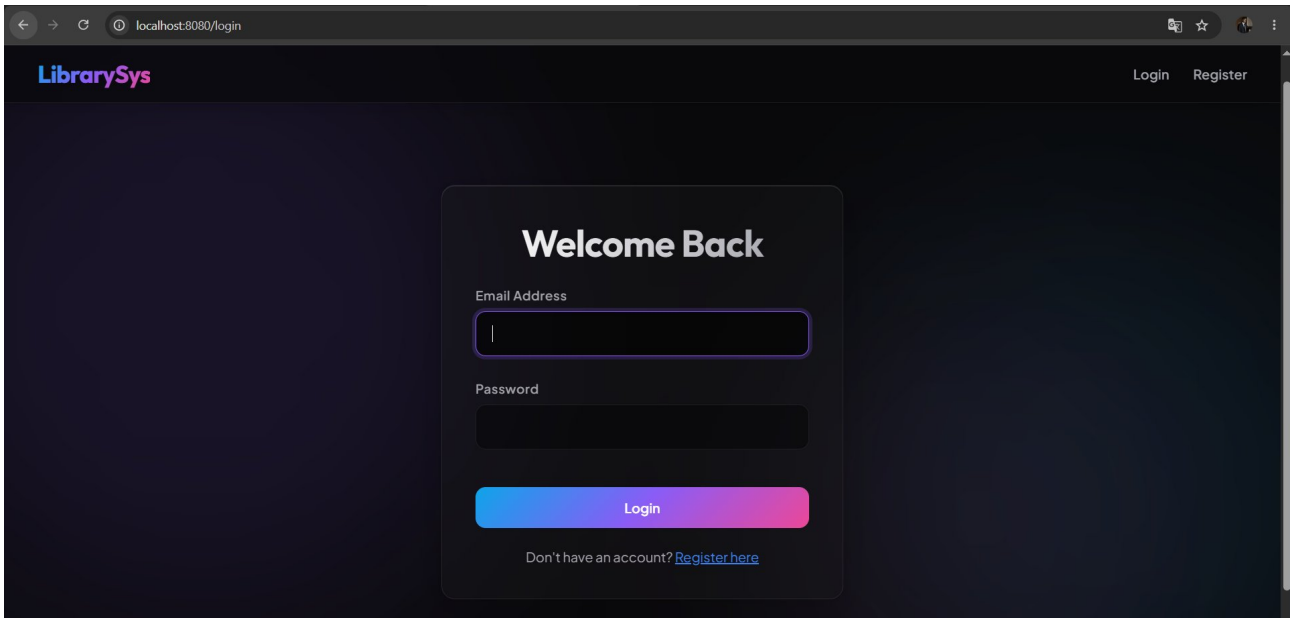


Figure 9. Trivy scan
0 vulnerabilities in the Alpine packages and 0 in the Composer packages.

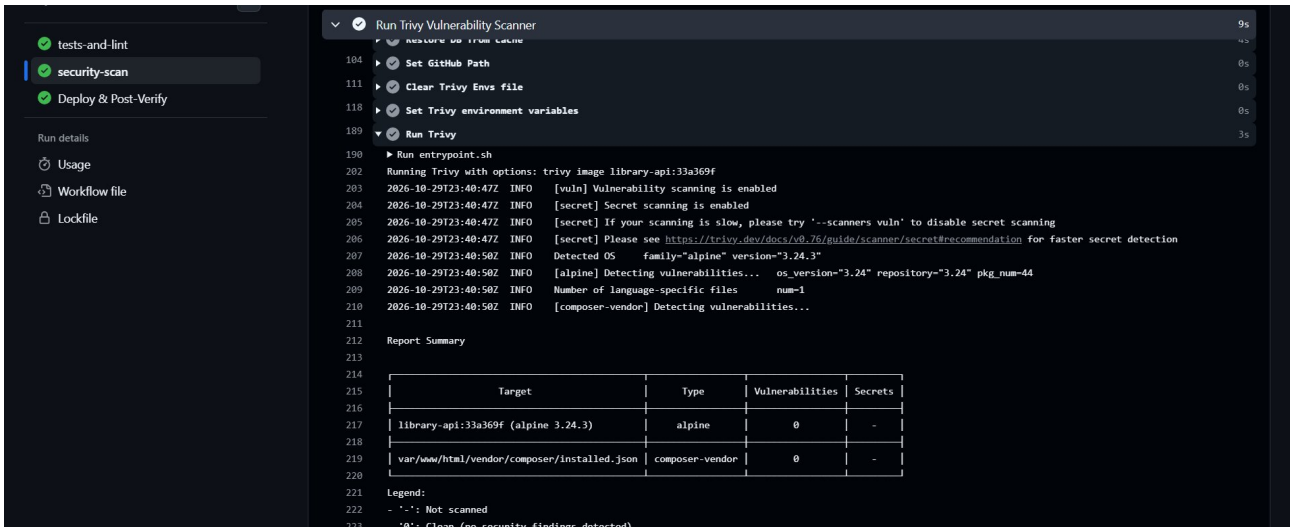


Figure 10. Composer audit
No security vulnerability advisories found.



Figure 11. Tests

7 tests passed.

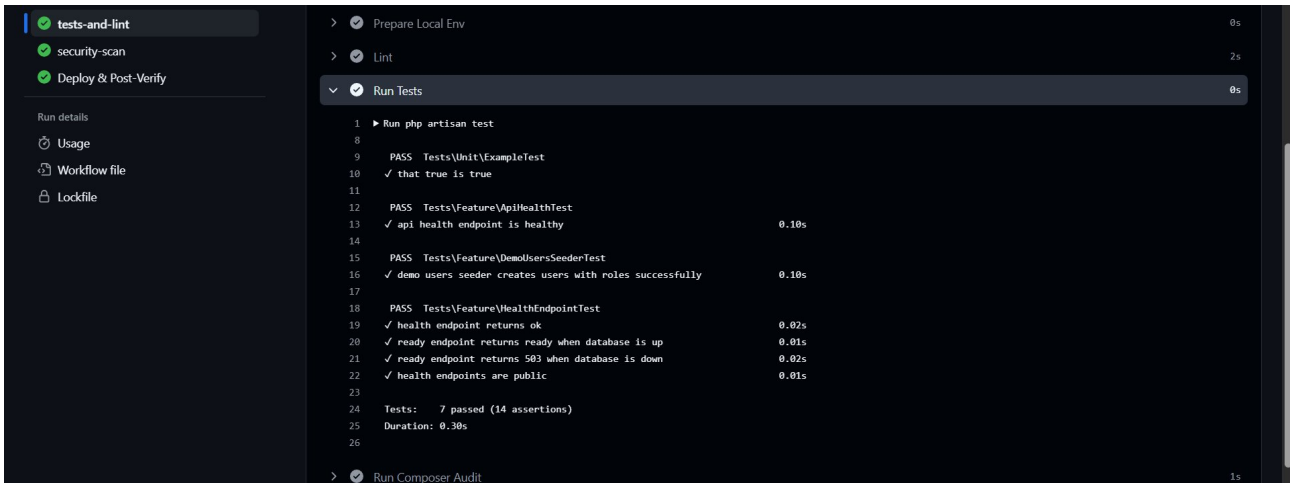


Figure 12. New release

Release v1.4 created after changing the application.

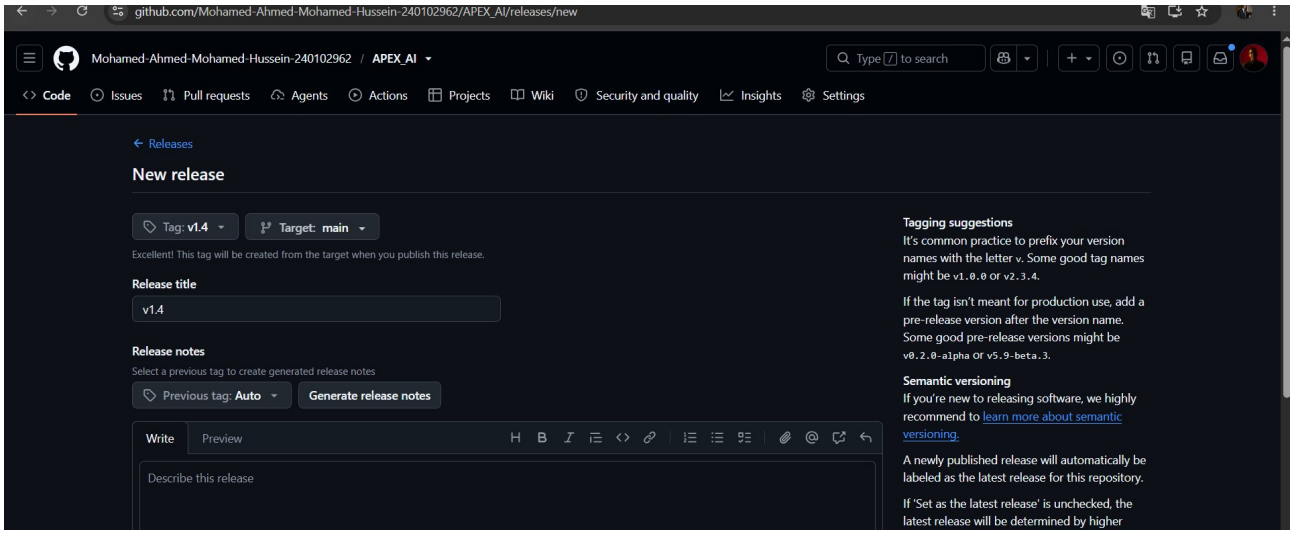


Figure 13. New version live

The dashboard on localhost:8080 shows the new version after the v1.4 deployment.

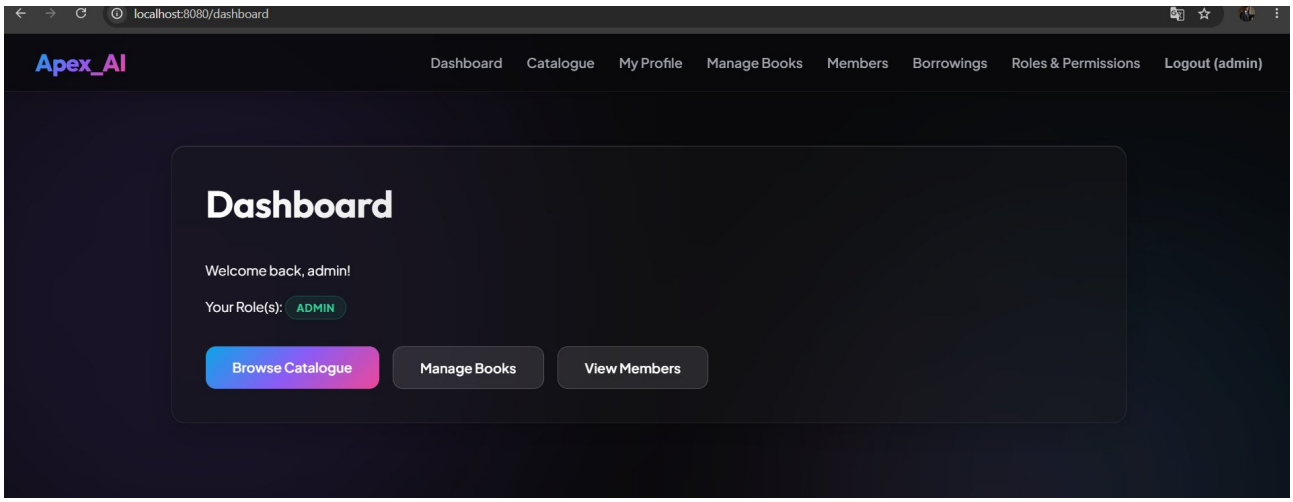


Figure 14. Run with caches

Release v1.4, run #16. All jobs passed in 1m 23s. The first run (figure 2) took 3m 37s.

